

lab_2-8

December 14, 2025

B.8. Implement KNN, Naïve Bayes, and SVM classifiers using Python-based ML tools for comparison of their performance on the review sentiment classification dataset.

```
[19]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from collections import Counter
#this is just to get data for SVM
from sklearn.datasets import make_blobs
```

KNN

Kth nearest neighbors(KNN) does not require any training In a way the dataset itself is the model. we are going to use the dataset itself and the nearest neighbors the predict which cluster out data belong to by calculating the euclidean distance from out point to the nearest points.

formula for euclidean distance on n dimensions are given below:

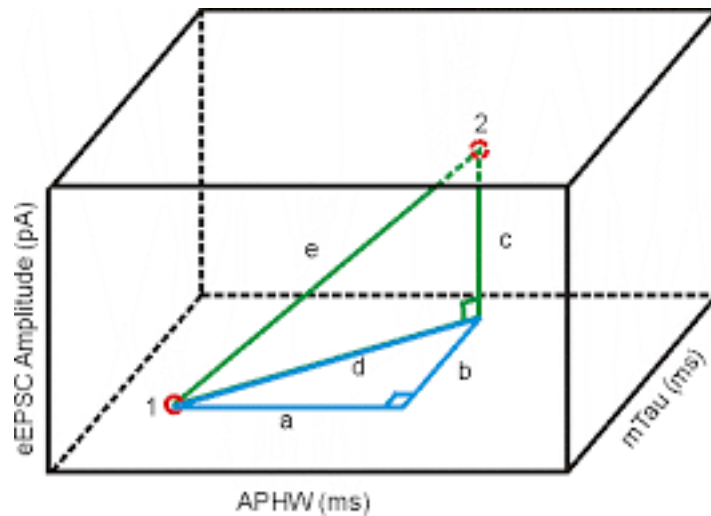
The Euclidean distance in n-dimensions is the straight-line distance between two points, generalized from the Pythagorean theorem: it's the square root of the sum of the squared differences between their corresponding coordinates across all n dimensions. For points $(P_1(x_1, x_2, \dots, x_n))$ and $(P_2(y_1, y_2, \dots, y_n))$, the formula is

$$d(p_1, p_2) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Formula

Points: $(P_1 = (x_1, x_2, \dots, x_n))$ and $(P_2 = (y_1, y_2, \dots, y_n))$.

Distance: $(d(P_1, P_2) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2})$



```
[12]: points = {
    'blue': [[2,4], [1,3], [2,3], [3,2], [2,1]],
    'red': [[5,6], [4,5], [4,6], [6,6], [5,4]]
}

new_point = [3,3]
```

```
[13]: def euclidean_dist(p,q):
    return np.sqrt(np.sum((np.array(p) - np.array(q))**2))
```

```
[14]: class knn:

    def __init__(self, k=3):
        self.k = k
        self.point = None

    def fit(self, points):
        self.points = points

    def predict(self, new_point):
        distances = []

        for category in self.points:
            for point in self.points[category]:
                distance = euclidean_dist(point, new_point)
                distances.append([distance, category])

        categories = [category[1] for category in sorted(distances)[:self.k]]
        result = Counter(categories).most_common(1)[0][0]
        return result
```

```
[15]: k = knn()  
      k.fit(points)
```

```
[16]: color = k.predict(new_point)
```

Visualization

```
[17]: ax = plt.subplot()  
      ax.grid(True, color='gray')  
      ax.set_facecolor('black')  
      ax.figure.set_facecolor('black')  
      ax.tick_params(axis="x", color='white')  
      ax.tick_params(axis="y", color='white')  
  
      for point in points['blue']:  
          ax.scatter(point[0], point[1], color='blue', s=60)  
  
      for point in points['red']:  
          ax.scatter(point[0], point[1], color='red', s=60)  
  
      ax.scatter(new_point[0], new_point[1], color=color, s=200, marker="*",  
                  ↪zorder=100)  
  
      for point in points['blue']:  
          ax.plot([new_point[0], point[0]], [new_point[1],point[1]], color='blue',  
                  ↪linestyle='--')  
  
      for point in points['red']:  
          ax.plot([new_point[0], point[0]], [new_point[1],point[1]], color='red',  
                  ↪linestyle='--')
```

